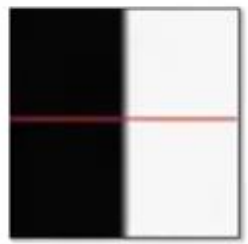
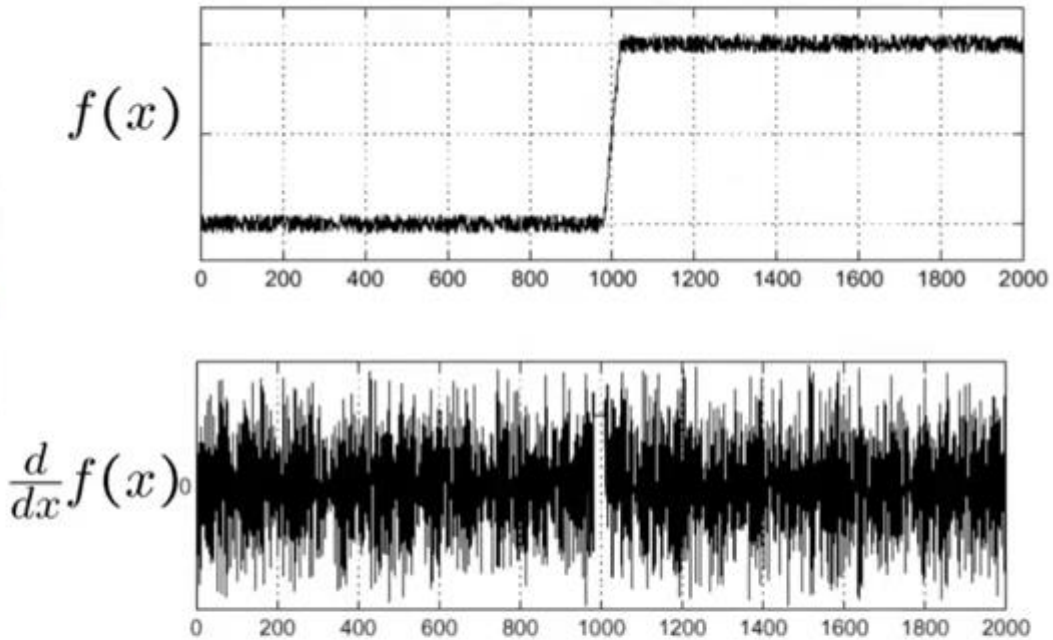
Edge Detection



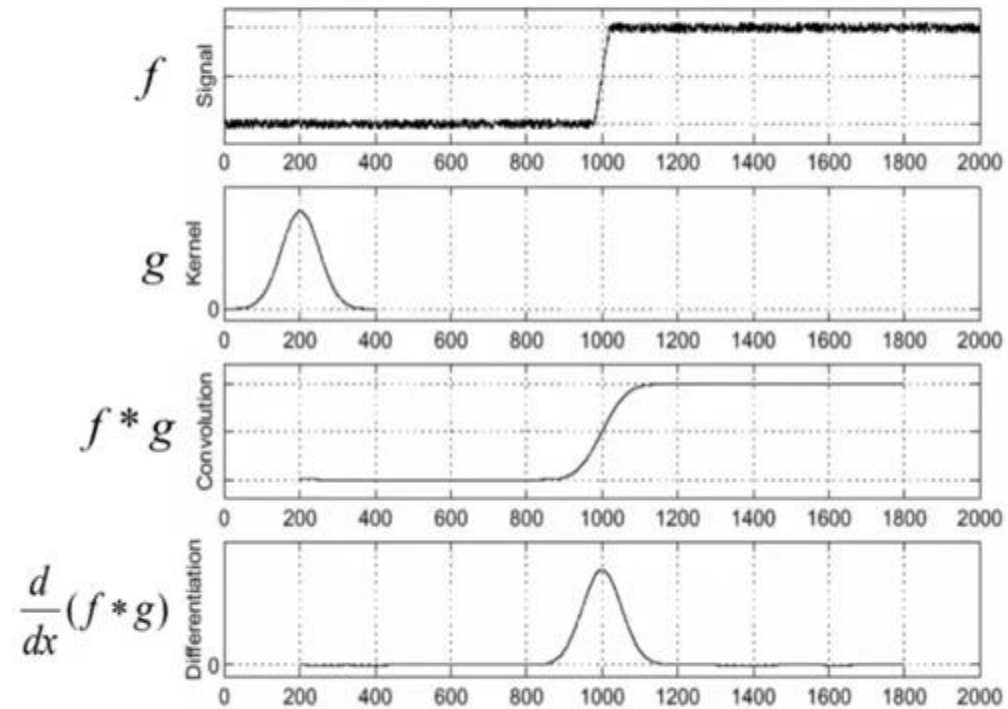
Noisy input image



Now, where is the edge?

Source: S Seitz, K Grauman

- Smooth first, and look for peaks in $\frac{d}{dx}(f * g)$

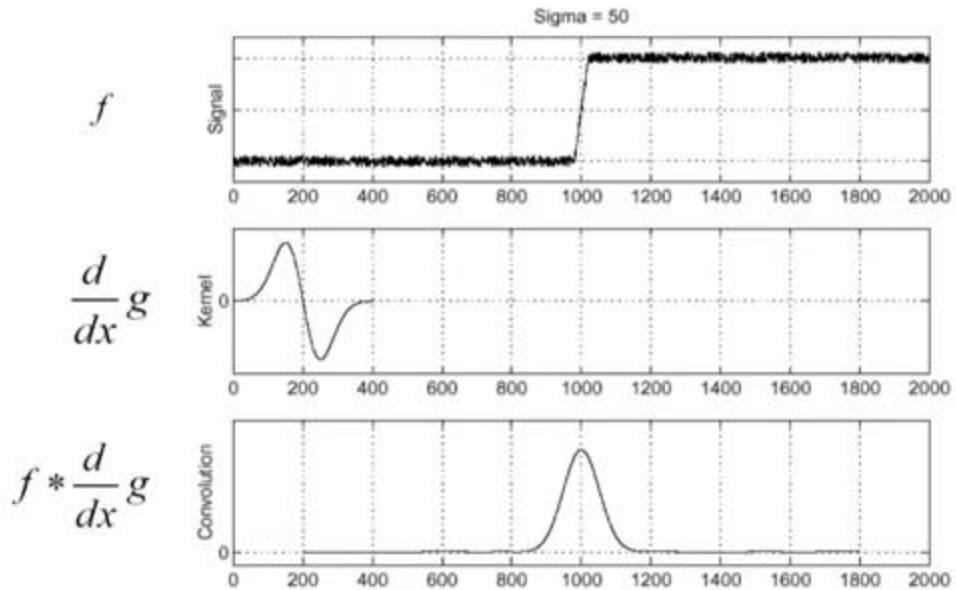


Source: S Seitz, R Urtasun

- Differentiation is achieved through convolution, and convolution is associative:

$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g = \frac{d}{dx}f * g$$

- This saves us an operation:



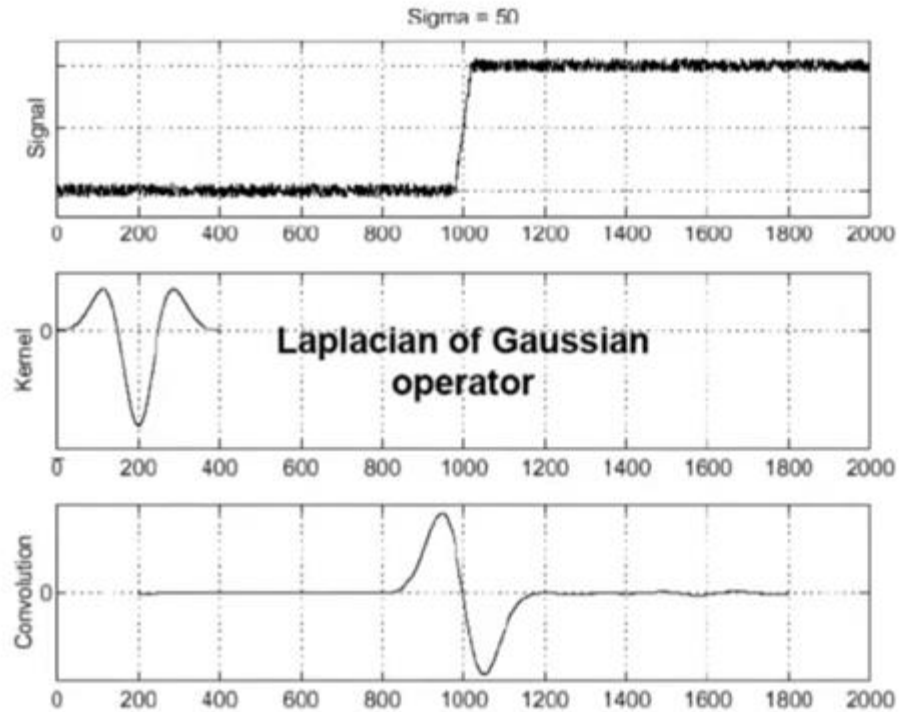
Source: S Seitz, R Urtasun

- Edge by detecting **zero-crossing** of bottom graph

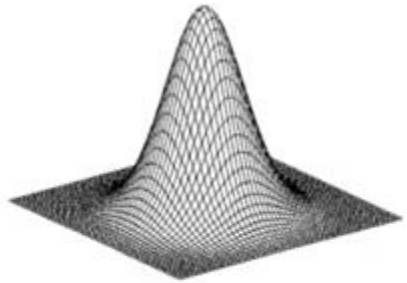
f

$\frac{\partial^2}{\partial x^2} h$

$(\frac{\partial^2}{\partial x^2} h) \star f$

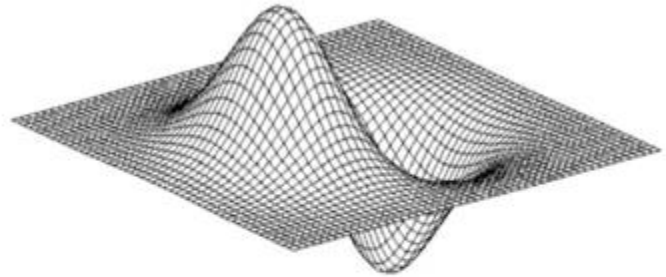


Source: S Seitz, R Urtasun



Gaussian

$h_\sigma(x,y) = \frac{1}{2\pi\sigma^2} \exp^{-\frac{u^2+v^2}{2\sigma^2}}$



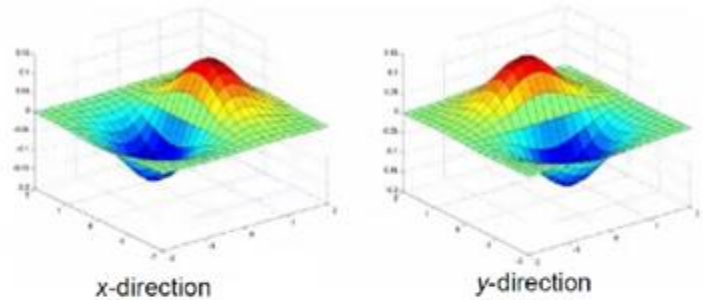
Derivative of Gaussian (x)

$\frac{\partial}{\partial x} h_\sigma(u,v)$



Laplacian of Gaussian

$\nabla^2 h_\sigma(u,v)$





with ∇^2 the Laplacian operator

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Which one finds horizontal/vertical edges?

Source: S Seitz, R Urtasun





X-Derivative of Gaussian



Y-Derivative of Gaussian

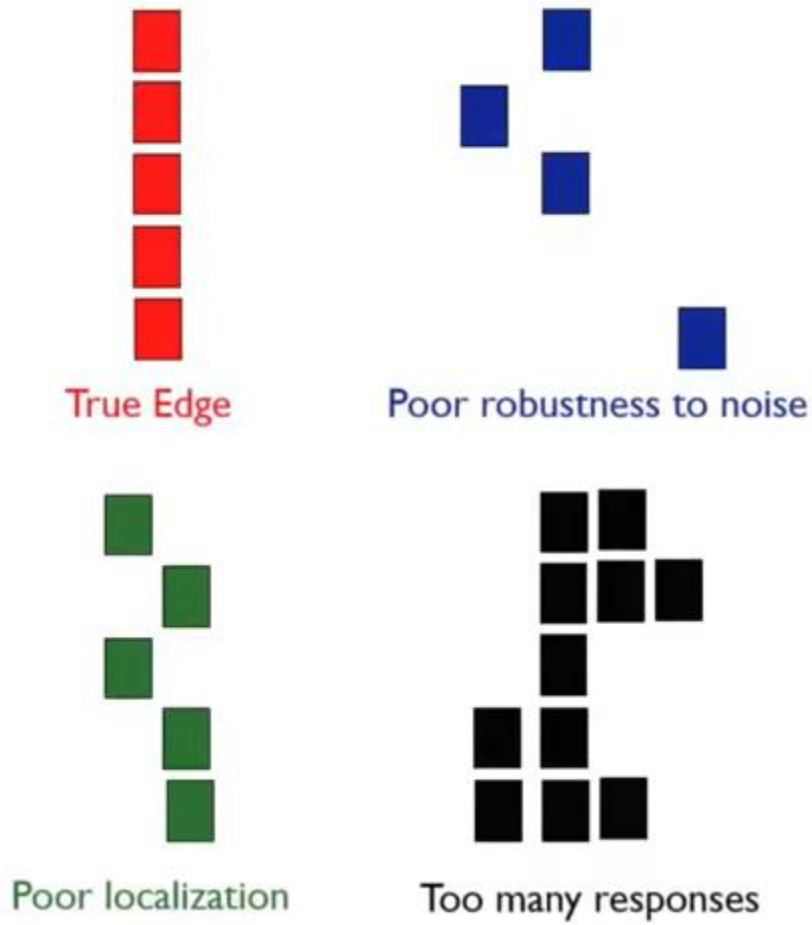


Gradient Magnitude

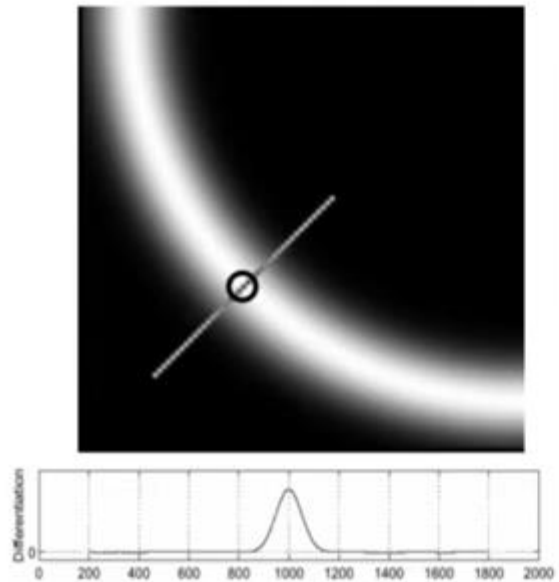

where is the edge?

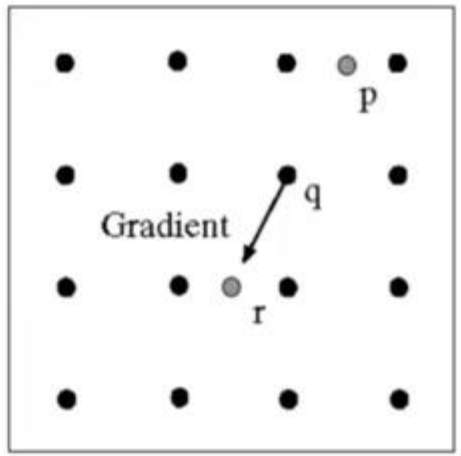
- Criteria for a good edge detector?

- Criteria for a good edge detector?
  - **Good detection:** find all real edges, ignoring noise or other artifacts
  - **Good localization:** detect edges as close as possible to true edges
  - **Single response:** return one point only for each true edge point









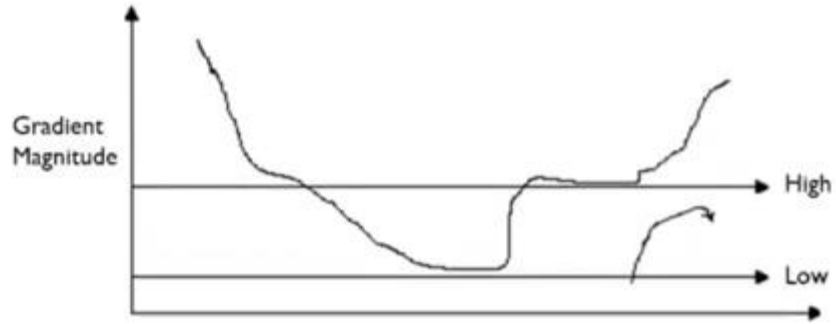
- Check if pixel is local maximum along gradient direction:
  - Could require checking interpolated pixels p and r



Before and after non-maxima suppression

- Check for well-connected edges. How?

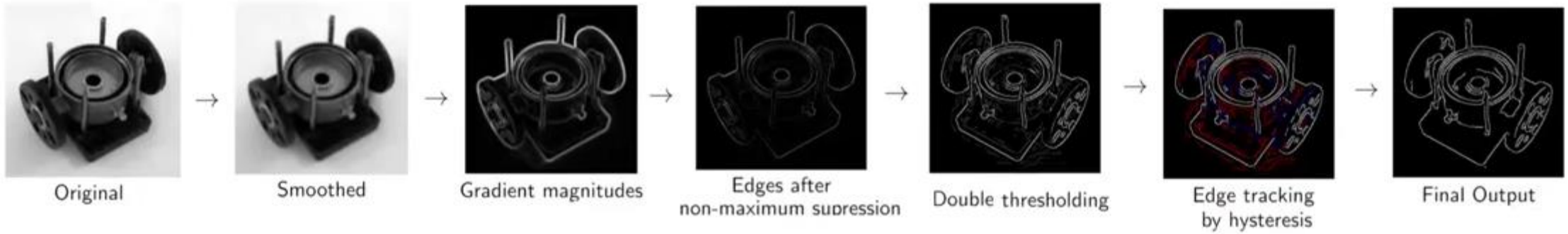
- Check for well-connected edges. How?
  - Use **hysteresis**: use a *high* threshold to start edge curves and a *low* threshold to continue them.



- How does it work?
  - If gradient at pixel $>$ 'High' $\Longrightarrow$ **'edge pixel'**
  - If gradient at pixel $<$ 'Low' $\Longrightarrow$ **'non-edge pixel'**
  - If gradient at pixel $\geq$ 'Low' and $\leq$ 'High' $\Longrightarrow$ **'edge pixel'** iff it is connected to an 'edge pixel' directly or via pixels between 'Low' and 'High'



- Probably the most widely used edge detector in computer vision (J Canny 1986 [1])
- Algorithm:
  1. Filter image with derivative of Gaussian
  2. Find magnitude and orientation of gradient
  3. Non-maximum suppression
  4. Linking and thresholding (hysteresis):
     - Define two thresholds: low and high
     - Use the high threshold to start edge curves and the low threshold to continue them


Original
Smoothed
Gradient magnitudes
Edges after
non-maximum supression
Double thresholding
Edge tracking
by hysteresis
Final Output



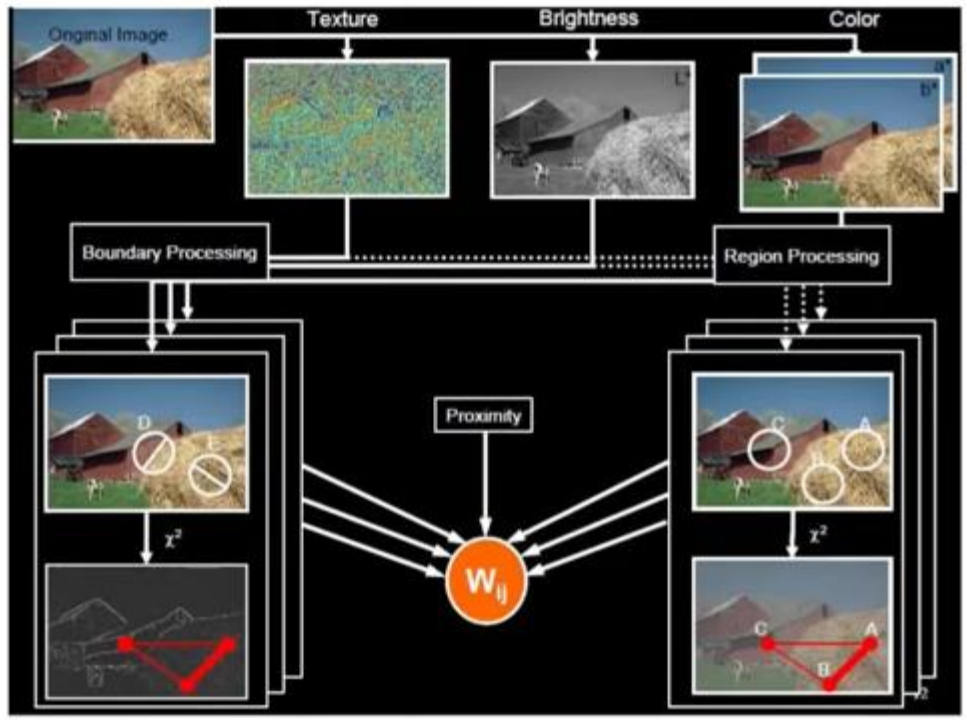
Canny Edges

- The choice of σ (Gaussian kernel spread/size) depends on desired behavior
  - large σ detects large-scale edges
  - small σ detects fine edges

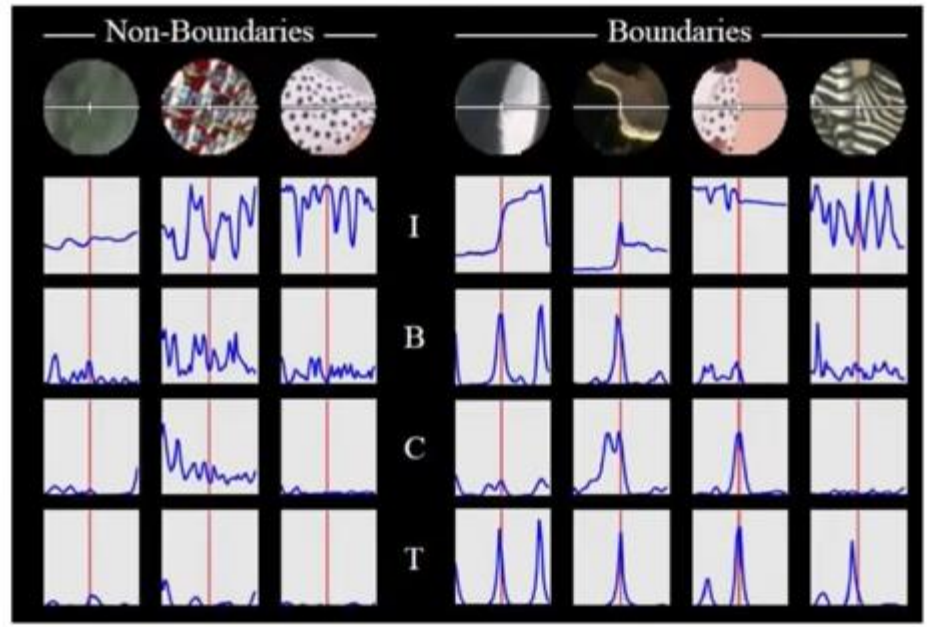


original Canny with $\sigma = 1$ Canny with $\sigma = 2$

Learning to Detect Natural Image Boundaries Using Local Brightness, Color, and Texture Cues (Martin et al. TPAMI 2004 [2])



pB Boundary Detector

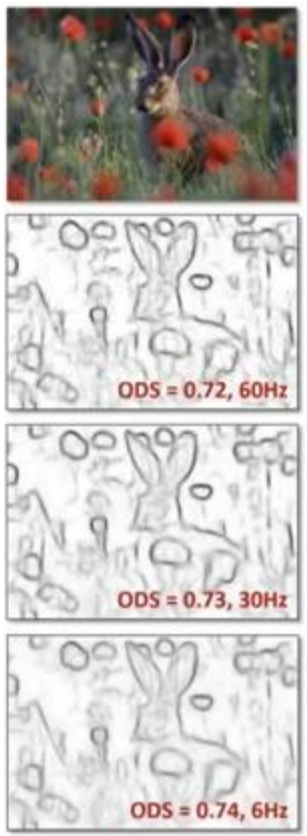


Learning to Detect Natural Image Boundaries Using Local Brightness, Color, and Texture Cues (Martin et al. TPAMI 2004 [2])



Structured Forests for Fast Edge Detection (Dollár et al. ICCV 2013 [3])

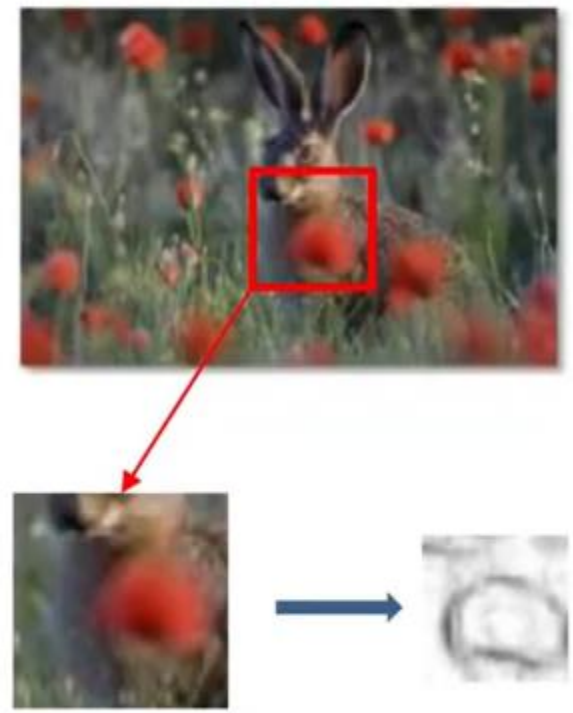
- Goal: quickly predict whether each pixel is an edge
- Insights
  - Predictions can be learned from training data
  - Predictions for nearby pixels should not be independent
- Solution
  - Train structured random forests to split data into patches with similar boundaries based on features
  - Predict boundaries at patch level, rather than pixel level, and aggregate (average votes)



Source: Derek Hoiem

Structured Forests for Fast Edge Detection (Dollár et al. ICCV 2013 [3])

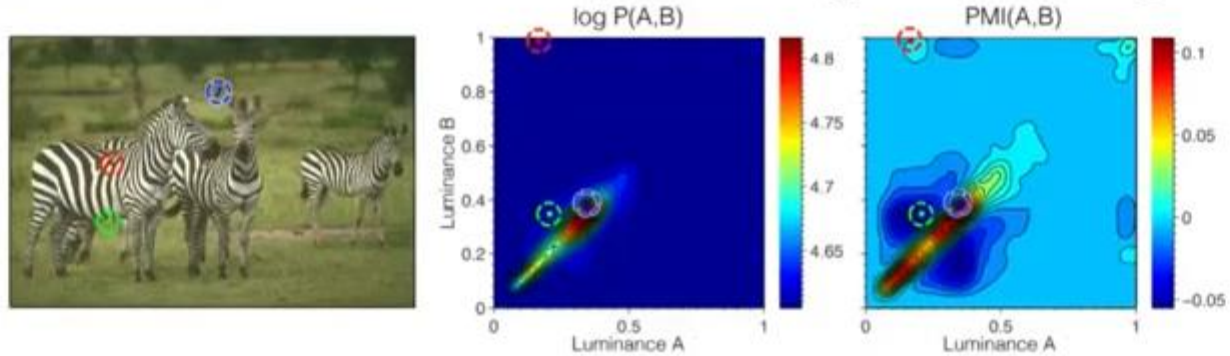
- Algorithm
  1. Extract overlapping 32×32 patches at three scales
  2. Features are pixel values and pairwise differences in feature maps (LUV color, gradient magnitude, oriented gradient)
  3. Predict T boundary maps in the central 16×16 region using T trained decision trees
  4. Average predictions for each pixel across all patches



Source: Derek Hoiem

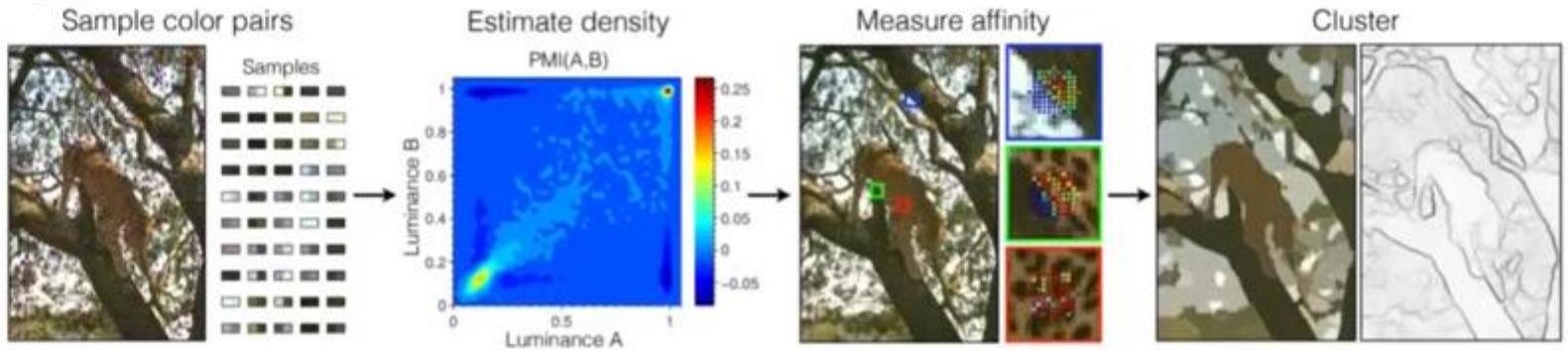
Crisp Boundary Detection using Pointwise Mutual Information (Isola et al. ECCV 2014 [4])

- Pixel combinations that are unlikely to be together are edges



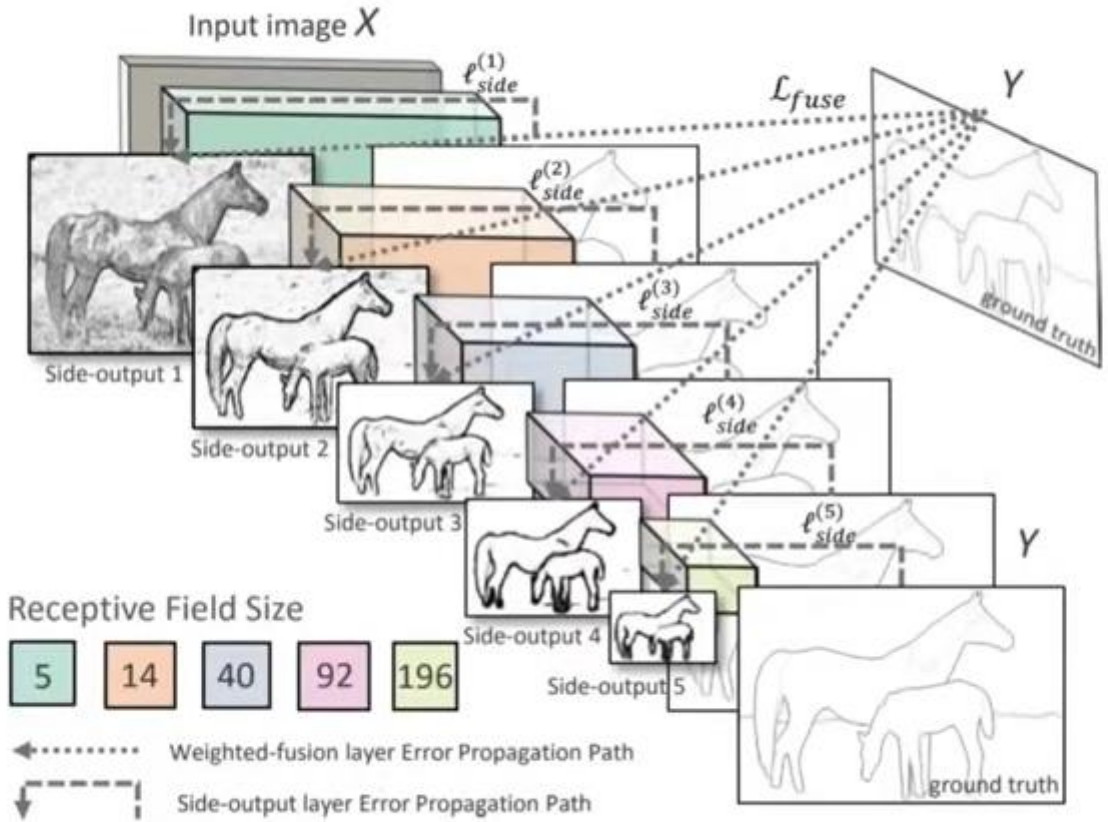
$$\mathrm{PMI}_\rho(A,B) = \log \frac{P(A,B)^\rho}{P(A)P(B)}$$

- Algorithm Pipeline:



Source: Derek Hoiem

Holistically Nested Edge Detection (Xie et al. ICCV 2015 [5])



	ODS	OIS	AP	FPS
Human	.80	.80	-	-
Canny	.600	.640	.580	15
Felz-Hutt [9]	.610	.640	.560	10
BEL [5]	.660*	-	-	1/10
gPb-owt-ucm [1]	.726	.757	.696	1/240
Sketch Tokens [24]	.727	.746	.780	1
SCG [31]	.739	.758	.773	1/280
SE-Var [6]	.746	.767	.803	2.5
OEF [13]	.749	.772	.817	-
DeepNets [21]	.738	.759	.758	1/5†
N4-Fields [10]	.753	.769	.784	1/6†
DeepEdge [2]	.753	.772	.807	$1/10^3$†
CSCNN [19]	.756	.775	.798	-
DeepContour [34]	.756	.773	.797	1/30†
HED (ours)	**.782**	**.804**	**.833**	2.5†, 1/12

Source: Derek Hoiem

Readings

- Chapter 2, Szeliski, *Computer Vision: Algorithms and Applications*

Questions

- How do you go from Canny edges to straight lines? *(Answer in next lecture)*





References

- https://www.youtube.com/watch?v=9r8ph2pb9aw